Export Chart Songs to an Apple Music Playlist

Context

Music Historeum currently only displays Billboard chart data — there's no way to act on it beyond viewing/paginating (the "create playlists" idea is currently just aspirational copy on `FeaturesPage.js`'s "Future Features" list). The user wants a real feature: pick a chart, select a number of songs from it (top 50, top 75, or a hand-picked set via checkboxes), and push those songs into a real Apple Music playlist in the user's own library.

This requires new integration with Apple's MusicKit API, which has two distinct trust boundaries:

- A **developer token** (signed server-side with an Apple-issued private key) that identifies *this app* to Apple.
- A **user token**, obtained client-side when the end user signs into their own Apple ID (must have an Apple Music subscription) via a MusicKit JS popup — our server is never involved in that step.

Blocking prerequisite: the user does not yet have an Apple Developer Program membership (paid, annual) or a MusicKit key. Before this feature can be tested end-to-end, they need to:

1. Enroll in the Apple Developer Program.
2. Under Certificates, Identifiers & Profiles → Keys, create a **MusicKit** key and download its `.p8` file (one-time download) and note its **Key ID**.
3. Note the account's **Team ID**.
4. Register the web origin(s) that will call MusicKit JS against the MusicKit identifier in the developer portal.

Everything below can be built now; only the last testing step is gated on the above.

Server changes (`server/index.js` only — stays a single file, matching current convention)

- Add `jsonwebtoken` as a new server dependency. Apple's ES256 developer-token JWT needs the raw JOSE (R||S) signature format, not the DER format Node's built-in `crypto.sign()` produces for EC keys — converting correctly is fiddly and error-prone to hand-roll, so use the standard library here rather than raw `crypto`.
- New env vars in `server/.env`: `APPLE_TEAM_ID`, `APPLE_KEY_ID`, `APPLE_PRIVATE_KEY_PATH`.
- Store the unmodified `.p8` file on disk at `server/keys/AuthKey_<KEYID>.p8` (referenced by `APPLE_PRIVATE_KEY_PATH`) rather than pasting a multi-line PEM into `.env` — avoids Windows CRLF/quoting issues. Add `keys/` to `server/.gitignore`.

- Near the existing nodemailer setup (`index.js:35-50`), add an analogous module-scope block: read the `.p8` file once at startup via `fs.readFileSync`, log success/failure with the existing `logger`. If the key isn't configured, log that Apple Music routes are disabled but don't crash the server — other routes must keep working before the Apple prerequisite is done.
- Add a small in-memory token cache: sign an ES256 JWT (`issuer: APPLE_TEAM_ID`, `keyid: APPLE_KEY_ID`, `expiresIn: 15777000` — Apple's ~6-month max) once, cache it, and only re-sign when it's within ~1 day of expiring.
- New route (alongside the other `app.get(...)` routes, after `/chart/:cid/:ctype/:ctf/:cdate`): `GET /apple-music/developer-token` → `503` JSON error if the key isn't loaded, otherwise `{ token }`; same try/catch + `logger.error` + `500` JSON shape as every other route.
- No CORS changes needed — the existing `cors({ origin: corsOrigins })` (`index.js:18`) already covers the new route.

Client changes

New service

- `client/src/services/fetchAppleMusicDeveloperToken.js` — mirrors `fetchContactForm.js`'s shape: `GET ${baseUrl}apple-music/developer-token`, throw on `!response.ok`, return `data.token`.

New feature directory: `client/src/features/appleMusicPlaylist/`

Kept separate from `services/` because most of this logic talks directly to Apple's client-side MusicKit API, not our Express backend.

- **`musicKitLoader.js`** — `loadMusicKit()`: singleton-promise loader that dynamically injects the MusicKit JS `<script>` tag (`https://js-cdn.music.apple.com/musickit/v3/musickit.js`) only the first time the feature is used (not added to `public/index.html` — most visitors won't use this, and it's third-party). Guards against double-injection and resolves on Apple's `musickitloaded` event.
- **`musicKitAuth.js`** — `getAuthorizedMusicKitInstance()`: loads MusicKit, fetches the developer token, calls `MusicKit.configure({ developerToken, app: { name: 'Music Historeum', build: '1.0.0' } })` once (cached), then `instance.authorize()` if not already authorized. A user declining sign-in should surface as a friendly error, not a crash.
- **`songMatcher.js`** — `matchSongsToAppleMusic(songs, { concurrency = 5, onProgress })`: for each `{song_title, artist_name}`, searches `/v1/catalog/{storefrontId}/search` (via `instance.api.music(...)`, which handles auth headers automatically), takes the top relevance result, and applies a normalized-substring-overlap check between our `artist_name` and the candidate's `attributes.artistName` to reject wildly wrong matches (mark as unmatched rather than force-adding). Runs with a small in-file concurrency-limited worker pool (~15 lines, no new dependency) and reports progress via `onProgress({ completed, total })`. Returns `{ matched: [{ song, appleMusicId }], unmatched: [song] }`.

- **`createAppleMusicPlaylist.js`** — `createAppleMusicPlaylist({ playlistName, songs, onProgress })`: authorizes, matches songs, `POST /v1/me/library/playlists` to create the playlist, then `POST /v1/me/library/playlists/{id}/tracks` to add matched tracks in one batch (fine up to Apple's ~100-item limit, which matches this feature's expected selection sizes). Returns a summary `{ playlistName, totalSelected, addedCount, unmatched }` for the UI to render.
- **`AppleMusicPlaylistToolbar.js`** — rendered only when `chart.chartType === 'Song'`. Shows selected count, Top-N quick buttons (50/75/100) plus a custom-count input, a "Clear selection" button, and "Create Apple Music Playlist" (disabled with no selection), which opens:
- **`CreatePlaylistModal.js`** — a `reactstrap` Modal following `ContactForm.js`'s existing modal pattern: playlist-name field (defaulted from the chart's title), a Create button driving authorize → search/match (with live progress) → create → add tracks → result summary (created, "X of N tracks added", list of unmatched titles).

`client/src/components/Table.js` — opt-in row selection, kept outside react-table's plugin state

Table.js currently has no row-selection plugin and only renders the paginated `page`, not full `rows`. Rather than wiring react-table's `useRowSelect` (which would need to reconcile selection across pagination/filtering), keep selection fully controlled by the parent — the same way the existing `artist_name` click-to-navigate special case (`Table.js:144-151, 233-254`) is handled outside the generic column path:

- New props: `selectable = false`, `selectedIds` (a `Set` of `song_id` strings), `onToggleRow(song_id)`.
- When `selectable`, prepend one checkbox `<th>`/`<td>` to the header row and each body row, reading `row.original.song_id` directly (works regardless of `hiddenColumns`).
- Selection identity is plain `song_id` strings owned by the parent, so it stays correct across page/filter changes with no react-table plugin-state complexity. No changes needed to the `*_COLUMNS.js` files.

`client/src/features/chart/ChartCard.js` — owns selection state, wires up the toolbar

- `const [selectedIds, setSelectedIds] = useState(() => new Set())`.
- `toggleRow(id)` — add/remove from the Set immutably.
- `selectTopN(n)` — `setSelectedIds(new Set(data.filter(r => r.song_rank <= n).map(r => String(r.song_id))))`. Filtering by `song_rank` on the full `data` array (already held here, `ChartCard.js:21`) is what makes "Top N" mean raw chart rank, independent of the table's current sort/filter/page.
- In the existing `useEffect` keyed on `[chart]` (`ChartCard.js:69-81`), reset `selectedIds` to empty whenever the chart changes.
- Pass `selectable={chartType === 'Song'}`, `selectedIds`, `onToggleRow={toggleRow}` into the existing `<Table .../>` call.
- Render `<AppleMusicPlaylistToolbar>` (only for Song charts) inside `CardBody`, passing `data`, `selectedIds`, `onSelectTopN={selectTopN}`, `onClear`, and a default playlist name derived from the

existing `chartTitle()`.

Sequencing

1. Server: dependency, env vars, `.gitignore`, `/apple-music/developer-token` route — independently testable via curl once Apple credentials exist.
2. Client selection UI (`Table.js` checkboxes, `ChartCard.js` selection state + Top-N): fully testable now, no Apple dependency at all.
3. Client MusicKit plumbing (`musicKitLoader.js`, `musicKitAuth.js`, token service): needs Apple prerequisite + a test Apple ID with an active Apple Music subscription to verify the `authorize()` popup and token round-trip.
4. Matching + playlist creation (`songMatcher.js`, `createAppleMusicPlaylist.js`, `CreatePlaylistModal.js`, `AppleMusicPlaylistToolbar.js`), wired into `ChartCard`.
5. Polish: progress/error copy, unmatched-song presentation, optional "select all on current page" convenience control.

Verification

- Steps 1-2 (server route, selection UI) can be verified immediately: `curl` the new route once configured; click through Top-N buttons and individual checkboxes on a live Song chart in the dev client and confirm the selected count and toolbar state update correctly, and that selection survives pagination/filtering/sort changes.
- Steps 3-4 require the user to complete the Apple Developer prerequisite and use a test Apple ID with an active Apple Music subscription: trigger the "Create Apple Music Playlist" flow from a real chart, confirm the Apple sign-in popup appears, confirm a playlist is actually created in the test account's Apple Music library with the expected tracks, and confirm the unmatched-songs list is sane by picking a chart with at least one hard-to-match/obscure title.